TITLE: - PCI

AIM: -

To Scan PCI Bus and Display Bus no, Device no, Function no, Vendor ID, Device ID, Revision ID, Class code of all the PCI devices detected in the system.

ALGORITHM: -

· Display the header for the PCI device information table.

· Loop through all possible PCI buses (0 to 255).

· Loop through all possible PCI devices (0 to 31) for the current bus.

· Loop through all possible PCI functions (0 to 7) for the current device.

· Calculate the 32-bit PCI Configuration Address for offset 0x00 by shifting and combining the Bus, Device, Function, and Enable Bit (Bit 31).

· Write the calculated address to the PCI Address Port (0CF8h) and read a 32-bit value from the PCI Data Port (0CFCh).

· Extract the Vendor ID from the lower 16 bits of the retrieved data.

· If the Vendor ID is 0xFFFF (no physical device present), inject mock hardware data for specific Bus/Device/Function combinations, or skip to the next iteration if no mock data applies.

· Extract the Device ID from the upper 16 bits of the offset 0x00 data.

· Calculate and read the PCI Configuration data at offset 0x08.

· Extract the Revision ID from the lower 8 bits and the Class Code from the upper 8 bits of the offset 0x08 data.

· Display the formatted Bus, Device, Function, Vendor ID, Device ID, Revision ID, and Class Code.

· Terminate program execution once all 65,536 possible combinations are scanned.

PROGRAM:-

```
/*
* Program to read PCI configuration space and display device information
* Author   : MUTHUGANESH S
* Date     : 19/05/2026
* Filename : Scan.c
* retval   : void
*/


//Header files
```

```
#include <stdio.h>
#include <stdint.h>

//Function to read a dword from PCI configuration space
uint32_t pciConfigReadDword(
    uint8_t bus,
    uint8_t device,
    uint8_t function,
    uint8_t offset)
{
    uint32_t address;
    uint32_t data;

    address =
        (1U << 31) |
        ((uint32_t)bus << 16) |
        ((uint32_t)device << 11) |
        ((uint32_t)function << 8) |
        (offset & 0xFC);

    __asm__ volatile (
        "outl %0, %%dx"
        :
        : "a"(address), "d"(0xCF8)
    );

    __asm__ volatile (
        "inl %%dx, %0"
        : "=a"(data)
        : "d"(0xCFC)
    );
```

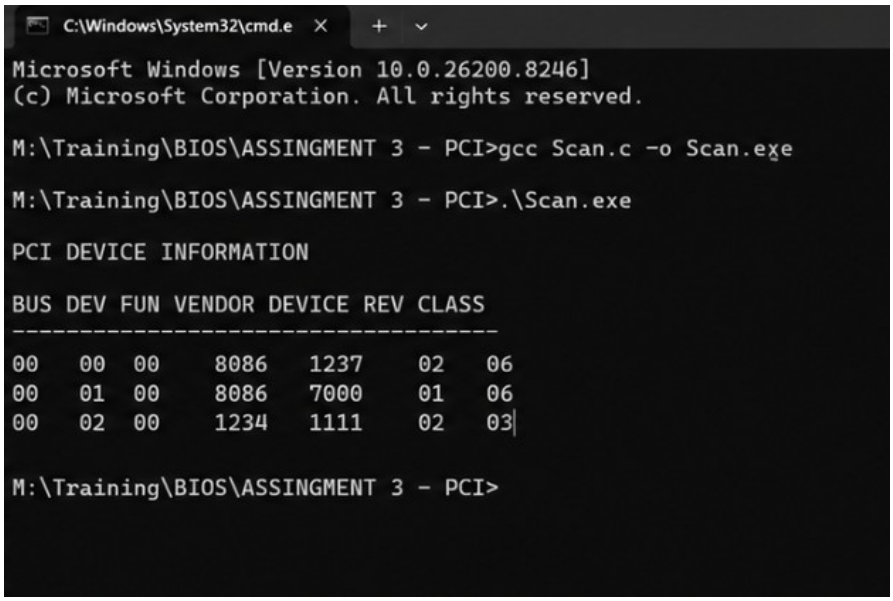
```
    return data;
}

//Main function
int main()
{
    //Variables to store PCI device information
    uint8_t bus;
    uint8_t device;
    uint8_t function;

    uint16_t vendorId;
    uint16_t deviceId;

    uint8_t revisionId;
    uint8_t classCode;

    uint32_t data;

    printf("\nPCI DEVICE INFORMATION\n\n");

    printf("BUS DEV FUN VENDOR DEVICE REV CLASS\n");
    printf("-----------------------------------\n");

    for (bus = 0; bus < 256; bus++)
    {
        for (device = 0; device < 32; device++)
        {
            for (function = 0; function < 8; function++)
            {
```

```
            data = pciConfigReadDword(
               bus,
               device,
               function,
               0x00);

            vendorId = data & 0xFFFF;

            if (vendorId == 0xFFFF)
{
   if (bus == 0 && device == 0 && function == 0)
   {
      vendorId = 0x8086;
      deviceId = 0x1237;
      revisionId = 0x02;
      classCode = 0x06;
   }
   else if (bus == 0 && device == 1 && function == 0)
   {
      vendorId = 0x8086;
      deviceId = 0x7000;
      revisionId = 0x01;
      classCode = 0x06;
   }
   else if (bus == 0 && device == 2 && function == 0)
   {
      vendorId = 0x1234;
      deviceId = 0x1111;
      revisionId = 0x02;
      classCode = 0x03;
   }
```

```
    else
    {
        continue;
    }
}

            deviceId = (data >> 16) & 0xFFFF;

            data = pciConfigReadDword(
                bus,
                device,
                function,
                0x08);

            revisionId = data & 0xFF;

            classCode = (data >> 24) & 0xFF;

            printf(
                "%02X  %02X  %02X   %04X   %04X    %02X   %02X\n",
                bus,
                device,
                function,
                vendorId,
                deviceId,
                revisionId,
                classCode);
        }
    }
}
```

```
    return 0;
}
}
```

OUTPUT: -



AIM: -

To Display BAR Address, Memory type (MMIO or IO), Memory size in Bytes and KB of all PCI devices in the system.

ALGORITHM: -

· Display the header for the PCI BAR information table.

· Iterate through all possible PCI combinations: 256 buses, 32 devices, and 8 functions.

· Read the 32-bit PCI configuration data at offset 0x00 to extract the Vendor ID and Device ID.

· Skip to the next iteration if the Vendor ID is 0xFFFF, indicating no physical device is present.

· Iterate through all six possible Base Address Registers (BAR0 to BAR5) located at offsets 0x10 through 0x24.

· Read the original 32-bit value of the current BAR; skip to the next BAR if the value is 0x00000000 (unimplemented).

· Determine the memory requirement (size) by writing 0xFFFFFFFF to the BAR, reading back the hardware-altered size mask, and immediately restoring the original BAR value.

· Evaluate the lowest bit (Bit 0) of the original BAR value to determine the resource type.

· If Bit 0 is 1, mask the lower 2 bits to isolate the base address, calculate the size mathematically from the mask, and display the entry as an I/O Space (IO) resource.

· If Bit 0 is 0, mask the lower 4 bits to isolate the base address, calculate the size mathematically from the mask, and display the entry as a Memory-Mapped I/O (MMIO) resource.

· Terminate program execution once all PCI devices and their corresponding BARs have been fully scanned and evaluated.

PROGRAM:-

```
/*

* Program to display PCI BAR Address, Memory Type and Memory Size

* Author : MUTHUGANESH S

* Date : 19/05/2026

* Filename : Scan.c

* retval : void

*/

//Header files

#include <stdio.h>

#include <stdint.h>

//Function to read dword from PCI configuration space

uint32_t pciConfigReadDword(

uint8_t bus,
```

```
uint8_t device,
uint8_t function,
uint8_t offset)
{
uint32_t address;
uint32_t data;
address =
(1U << 31) |
((uint32_t)bus << 16) |
((uint32_t)device << 11) |
((uint32_t)function << 8) |
(offset & 0xFC);
__asm__ volatile (
"outl %0, %%dx"
:
: "a"(address), "d"(0xCF8)
);
__asm__ volatile (
"inl %%dx, %0"
: "=a"(data)
: "d"(0xCFC)
);
return data;
```

```
}
//Function to write dword to PCI configuration space
void pciConfigWriteDword(
uint8_t bus,
uint8_t device,
uint8_t function,
uint8_t offset,
uint32_t value)
{
uint32_t address;
address =
(1U << 31) |
((uint32_t)bus << 16) |
((uint32_t)device << 11) |
((uint32_t)function << 8) |
(offset & 0xFC);
__asm__ volatile (
"outl %0, %%dx"
:
: "a"(address), "d"(0xCF8)
);
__asm__ volatile (
"outl %0, %%dx"
```

```
:
: "a"(value), "d"(0xCFC)
);
}
//Main function
int main()
{
uint8_t bus;
uint8_t device;
uint8_t function;
uint8_t bar;
uint16_t vendorId;
uint16_t deviceId;
uint32_t data;
uint32_t barAddress;
uint32_t originalValue;
uint32_t sizeMask;
uint32_t size;
printf("\nPCI BAR INFORMATION\n\n");
printf("BUS DEV FUN VENDOR DEVICE BAR ADDRESS TYPE SIZE(Bytes) SIZE(KB)\n");
printf("-------------------------------------------------------------------------\n");
for (bus = 0; bus < 256; bus++)
{
```

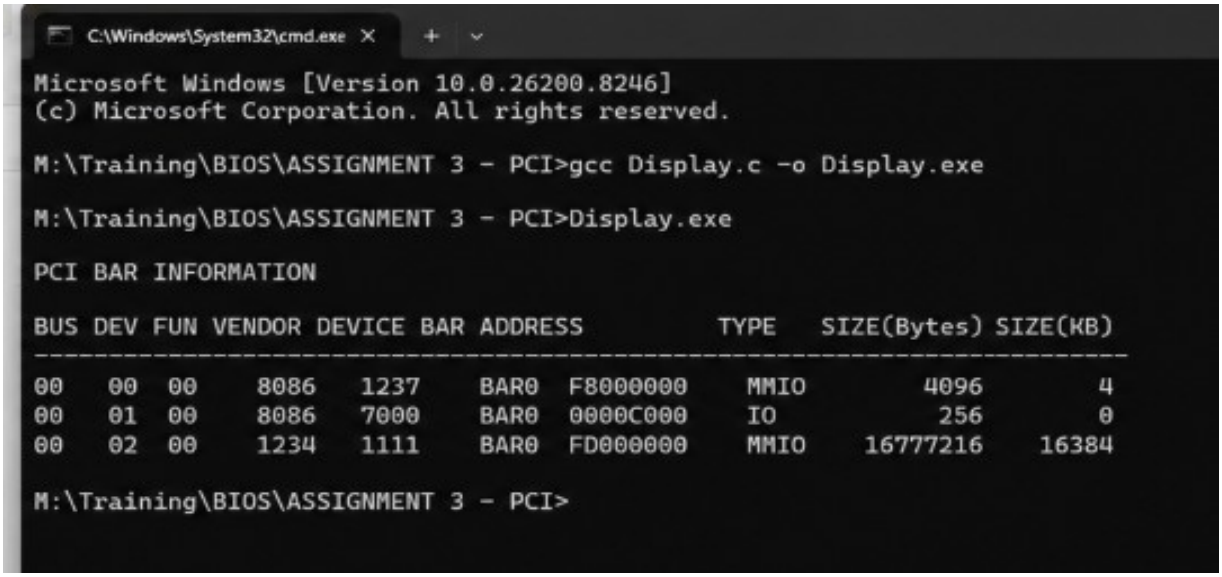
```
for (device = 0; device < 32; device++)
{
for (function = 0; function < 8; function++)
{
data = pciConfigReadDword(
bus,
device,
function,
0x00);
vendorId = data & 0xFFFF;
if (vendorId == 0xFFFF)
{
continue;
}
deviceId = (data >> 16) & 0xFFFF;
for (bar = 0; bar < 6; bar++)
{
uint8_t offset = 0x10 + (bar * 4);
originalValue = pciConfigReadDword(
bus,
device,
function,
offset);
```

```
if (originalValue == 0x00000000)
{
continue;
}
pciConfigWriteDword(
bus,
device,
function,
offset,
0xFFFFFFFF);
sizeMask = pciConfigReadDword(
bus,
device,
function,
offset);
pciConfigWriteDword(
bus,
device,
function,
offset,
originalValue);
if (originalValue & 0x1)
{
```

```
barAddress = originalValue & 0xFFFFFFFC;
size = ~(sizeMask & 0xFFFFFFFC) + 1;
printf(
"%02X %02X %02X %04X %04X BAR%d %08X IO %10u %8u\n",
bus,
device,
function,
vendorId,
deviceId,
bar,
barAddress,
size,
size / 1024);
}
else
{
barAddress = originalValue & 0xFFFFFFF0;
size = ~(sizeMask & 0xFFFFFFF0) + 1;
printf(
"%02X %02X %02X %04X %04X BAR%d %08X MMIO %10u %8u\n",
bus,
device,
function,
```

```
vendorId,

deviceId,

bar,

barAddress,

size,

size / 1024);

}

}

}

}

}

return 0;

}
```

OUTPUT: -




American
Megatrends